专利：A GPU-Oriented Hardware Architecture and Method for Prefetching Data from CPU Memory to GPU HBM

Abstract

A system and method are disclosed for optimizing data prefetching in a parallel processing unit, particularly for models employing a Mixture of Experts (MoE) architecture during inference. In one embodiment, the system includes a dedicated prefetch processor integrated within a parallel processing unit (such as a GPU), configured to execute a lightweight prefetch program. The prefetch processor performs simple predictive computations to predict data (e.g., MoE expert parameters) that will be required for subsequent execution, and issues read commands to prefetch the predicted data from CPU memory to high-bandwidth memory (HBM) of the parallel processing unit. By prefetching only the required data (instead of all data) into HBM, the system reduces HBM storage overhead, improves memory utilization, and enhances the execution efficiency of MoE inference and other data-dependent workloads. The disclosed prefetch hardware is versatile and can be applied to various scenarios requiring advance data prefetching, beyond MoE inference optimization.

Background

Parallel processing units, such as Graphics Processing Units (GPUs), are widely used for accelerating the computation of Large Language Models (LLMs), including various complex inference and fine-tuning tasks. A critical challenge in LLM computation is the efficient utilization of high-bandwidth memory (HBM) on GPUs, as LLMs typically involve large-scale data (e.g., model parameters, intermediate activation values, task-specific adapters) that far exceeds the limited capacity of HBM. In traditional LLM computation architectures, there is a lack of a dedicated hardware mechanism to prefetch only the data required for subsequent execution, leading to the common practice of loading all potentially needed data into HBM in advance—this results in severe HBM storage waste, as only a subset of the loaded data is actually accessed during execution. This problem is prevalent across multiple LLM computation scenarios, and a dedicated prefetch hardware is urgently needed to address this inefficiency.

For instance, in Mixture of Experts (MoE) models, multiple expert sub-networks are integrated to achieve high model capacity, but each input token (or sequence of tokens) only activates a small subset of the experts during inference. Conventional MoE inference implementations load all

expert parameters into GPU HBM in advance due to the absence of a predictive prefetch mechanism, leading to significant underutilization of HBM resources. In LLM fine-tuning with LoRA (Low-Rank Adaptation), a large number of task-specific LoRA adapter parameters are often stored in CPU memory, but only a subset of these adapters are activated for a specific input task or prompt; loading all LoRA adapters into HBM in advance wastes valuable storage space. Another typical scenario is LLM context window management: during long-sequence inference (e.g., context windows of 100k tokens or more), the full context sequence and its associated intermediate activation values are typically stored in CPU memory, but only a contiguous segment of the context (e.g., the most recent 2k tokens) is required for the current inference step.

HBM is a high-cost, high-bandwidth memory with limited capacity; the waste of HBM space in these LLM scenarios not only increases hardware costs but also may lead to memory bottlenecks when multiple tasks are executed concurrently, thereby reducing the overall LLM computation throughput. A dedicated prefetch hardware that can predict and prefetch only the required data from CPU memory to GPU HBM is therefore essential to solve this widespread problem.

Existing data prefetching techniques primarily rely on software-based prefetching or general-purpose processor-controlled prefetching. Software-based prefetching requires manual code optimization by developers, which is time-consuming and lacks generality—different workloads require customized prefetch logic. General-purpose processor (e.g., CPU or GPU core) controlled prefetching occupies valuable computing resources that could otherwise be used for core inference tasks, leading to reduced execution efficiency. Additionally, these techniques cannot efficiently handle the dynamic, data-dependent prefetching requirements of MoE inference, where the required expert parameters change with each input token.

There is therefore a need for a dedicated, versatile hardware mechanism that can efficiently perform predictive data prefetching, particularly for MoE inference scenarios, to reduce HBM storage overhead, improve memory utilization, and avoid occupying core computing resources. This mechanism should be capable of executing lightweight predictive computations and issuing prefetch commands independently, without relying on intervention from a host processor or core computing units.

Summary of the Invention

The present invention introduces a dedicated prefetch hardware architecture that enables efficient, predictive data prefetching for parallel processing units, addressing the HBM storage waste problem and providing a versatile solution for various prefetching scenarios. The invention is based on integrating a dedicated prefetch processor within a parallel processing unit (e.g., GPU), where the prefetch processor is configured to execute a lightweight prefetch

program, perform predictive computations, and issue data read commands to prefetch required data from CPU memory to the parallel processing unit's HBM.

In one embodiment, the prefetch processor includes a lightweight scalar processing core, a prefetch instruction memory, a prediction logic module, and a command interface. The prefetch instruction memory stores a prefetch program tailored to specific workloads (e.g., MoE expert activation prediction), which includes logic for performing simple predictive computations (e.g., predicting activated experts based on input token features). The prediction logic module executes the prefetch program to generate predictions of the data that will be required for subsequent execution (e.g., the set of MoE experts that will be activated by the next input token).

Based on the prediction results, the prefetch processor issues read commands via the command interface to transfer the predicted required data (e.g., parameters of the predicted activated experts) from CPU memory to the HBM of the parallel processing unit. This ensures that only the data needed for subsequent execution is loaded into HBM, rather than all data, thereby reducing HBM storage overhead and improving memory utilization.

The disclosed prefetch hardware can be configured with different prefetch programs to support various data-dependent workloads that require advance data prefetching, such as other neural network architectures, batch processing tasks, or any application where data requirements can be predicted with simple computations. The prefetch processor operates independently of the parallel processing unit's core computing units (e.g., stream multiprocessors in GPUs) and host processors, avoiding the occupation of core computing resources and reducing host-device interaction overhead.

Detailed Description

Overall System Architecture

In one embodiment, a processing system comprises a host processor (e.g., CPU) with host memory (CPU memory), a parallel processing unit (e.g., GPU) with high-bandwidth memory (HBM), and a dedicated prefetch processor integrated within the parallel processing unit. The parallel processing unit further includes core computing units (e.g., stream multiprocessors (SMs) in GPUs) configured to execute main workloads, a memory controller for managing data transfers between HBM and the core computing units, and a bus interface for communicating with the host processor.

The host memory stores large-scale data that may be required for execution (e.g., all expert parameters of an MoE model, all tokens for overlong-context text). The HBM of the parallel processing unit stores the data that is currently required or will be required imminently for execution. The prefetch processor is integrated between the bus interface and the memory controller, enabling it to independently issue read commands to transfer data from host memory to HBM.

The core computing units (e.g., SMs) execute the main workload using the data stored in HBM. Unlike conventional systems where all data is loaded into HBM in advance, the core computing units in the disclosed system only need to access the prefetch data stored in HBM, reducing HBM storage usage and improving memory access efficiency.

Prefetch Processor Architecture

In one embodiment, the prefetch processor comprises a lightweight scalar processing core, a prefetch instruction memory, a prediction logic module, a command generator, and a control register file. The prefetch processor is designed to be lightweight, with a simplified instruction set tailored to predictive computations and prefetch command generation, ensuring it does not occupy excessive hardware resources.

The prefetch instruction memory stores one or more prefetch programs, which are configurable based on the target workload. The prefetch program includes logic for performing fast, lightweight predictive computations—specifically, a small number of matrix multiplications and non-linear operations—to predict the data that will be required for subsequent execution of the core computing units. The scalar processing core executes the prefetch program stored in the prefetch instruction memory. It includes a scalar arithmetic logic unit (ALU) for performing simple arithmetic and logical operations required for prediction, and a control unit for fetching and decoding prefetch instructions. The control register file stores intermediate values generated during the predictive computation (e.g., intermediate results of the lightweight computations, prediction scores) and configuration parameters (e.g., prefetch threshold, data transfer batch size, number of consecutive computation stages to predict).

The prediction logic module is coupled to the scalar processing core and is configured to process the results of the predictive computation to determine the exact data to be prefetched. This module converts the prediction results into the memory addresses of the corresponding required data in host memory. The command generator is configured to generate read commands based on the memory addresses, which include the source address (in host memory), destination address (in HBM), and data size.

Prefetch Process and Interaction Mechanism

The prefetch process of the disclosed system is divided into three main stages: predictive computation, command generation, and data transfer. These stages are executed independently by the prefetch processor, without intervention from the host processor or the parallel processing unit's core computing units.

In the first stage (predictive computation), the prefetch processor receives input data related to the main workload—this input data is the intermediate computation result generated by the core computing units during their current execution (e.g., intermediate representation of data being processed by the core computing units). The scalar processing core executes the prefetch

program to perform fast, lightweight predictive computations on this input data; these computations may include a small number of matrix multiplications and non-linear operations, which are tailored to predict the data required for the core computing units' subsequent execution.

In the second stage (command generation), the prediction logic module converts the prediction results into memory addresses corresponding to the data required for the core computing units' subsequent execution. The command generator then generates read commands, which specify the source address (host memory address of the required data), destination address (HBM address where the data will be stored), and the size of the data to be transferred.

In the third stage (data transfer), the prefetch processor sends the generated read commands to the memory controller of the parallel processing unit. The memory controller communicates with the host processor via the bus interface to transfer the specified data from host memory to the target address in HBM. Once the data transfer is complete, the prefetch processor updates the control register file to indicate that the data is ready for use by the core computing units.

The prefetch processor operates in parallel with the core computing units in a stage-synchronized manner. While the core computing units are executing the current stage of the main workload, the prefetch processor uses the intermediate result of this current stage to perform predictive computations for the core computing units' subsequent execution stages, and initiates the prefetching of the required data. This overlap between the core computing units' current stage computation and the prefetch processor's predictive prefetching for subsequent stages eliminates the latency associated with data transfer, further improving execution efficiency.

Example: MoE Inference Optimization

In one embodiment, the disclosed system is used to optimize the inference of an MoE model. The MoE model includes a plurality of expert sub-networks and is divided into multiple sequential layers; during inference, each input token (or sequence of tokens) only activates a small subset of experts in each layer, and the activated experts vary across different layers. All expert parameters of the MoE model are stored in CPU memory, and the prefetch processor is configured with a dedicated prefetch program tailored to MoE expert activation prediction—this program is designed to perform fast, lightweight computations to predict expert activation across consecutive layers.

During inference initialization, the host processor loads the prefetch program into the prefetch instruction memory of the prefetch processor and configures key prefetch parameters, including the number of consecutive layers (denoted as k) for which expert activation is to be predicted (e.g., k=2 or k=3, meaning predicting expert activation for layers j to j+k when the GPU is processing layer j of a token), the number of experts to predict per layer, and the similarity threshold for activation. The core computing units (e.g., GPU SMs) begin processing the t-th

input token, starting with the first layer (j=1); as the core computing units compute the representation of the t-th token at layer j, they transmit this layer-j representation to the prefetch processor in real time.

While the core computing units are continuing to process the t-th token at layer j (i.e., completing the computation of layer j and preparing to enter layer j+1), the prefetch processor receives the layer-j representation of the t-th token and executes the prefetch program to perform fast predictive computations. These fast computations include a small number of matrix multiplications and non-linear operations (e.g., ReLU activation), which are lightweight and do not occupy core computing resources. Based on the results of these computations, the prefetch processor predicts the set of experts that will be activated by the t-th token in layers j to j+k (consecutive k layers following the current layer j). After determining the predicted activated experts for layers j to j+k, the prefetch processor converts the identifiers of these experts into their corresponding parameter memory addresses in CPU memory, and generates read commands to transfer the parameters of these predicted experts from CPU memory to GPU HBM.

When the core computing units finish processing the t-th token at layer j and move to layer j+1 (the first layer in the predicted j~j+k range), the parameters of the experts required for layer j+1 have already been prefetched into HBM. This process repeats iteratively every k layers of the t-th token: the prefetch processor always uses the current layer's representation of the t-th token to predict expert activation for the next k consecutive layers, and prefetches the required expert parameters in advance. This ensures that the core computing units always have immediate access to the expert parameters required for the current and subsequent k layers in HBM, without waiting for data transfer from CPU memory.

By only prefetching the parameters of the experts predicted to be activated for the next k consecutive layers (instead of all expert parameters or even all experts for a single layer in advance), the system significantly reduces HBM storage usage. The reduction ratio depends on the total number of experts in the MoE model, the number of activated experts per layer, and the value of k—for example, a 60-expert MoE model with 4 activated experts per layer and k=6 (predicting 6 consecutive layers) over all 24 layers, reduces HBM usage by approximately (60 ÷ 4 experts) × (24 ÷ 6 layers) = 60 times.

Alternative Embodiments

The prefetch processor may be implemented in various forms. In some embodiments, it may be a programmable microcontroller with a simplified instruction set, allowing for flexible configuration of prefetch programs for different workloads. In other embodiments, it may be a microcoded sequencer that executes predefined prefetch logic for specific scenarios (e.g., MoE inference, batch processing). In still other embodiments, it may be implemented as a finite state machine (FSM) optimized for a single type of prefetch task, reducing hardware complexity.

The prefetch processor may include multiple prediction logic modules to support concurrent prefetching for multiple workloads. For example, it may simultaneously predict expert activation for MoE inference and prefetch data for another batch processing task. Additionally, the prefetch processor may include a feedback mechanism that adjusts the prediction logic based on actual data usage (e.g., if a predicted expert is not activated, the similarity threshold is adjusted to improve prediction accuracy).

The prefetch program stored in the prefetch instruction memory may be updated dynamically. The host processor or core computing units may send updated prefetch logic to the prefetch processor during execution, allowing the system to adapt to changing workload requirements. For example, in MoE inference, if the model's expert activation pattern changes (e.g., due to a shift in input data distribution), the prefetch program can be updated to maintain prediction accuracy.

Claims

What is claimed is:

1. A processing system comprising a host processor with host memory, a parallel processing unit with high-bandwidth memory (HBM) and core computing units configured to execute main workloads, and a dedicated prefetch processor integrated within the parallel processing unit; wherein the prefetch processor is configured to execute a prefetch program to perform predictive computations, predict data required for subsequent execution of the core computing units, and issue read commands to prefetch the predicted data from the host memory to the HBM, without requiring intervention from the host processor or the core computing units.
2. The system of claim 1, wherein the main workload includes inference of a Mixture of Experts (MoE) model, and the predicted data includes parameters of experts in the MoE model that are predicted to be activated during inference.
3. The system of claim 1, wherein the prefetch processor comprises a scalar processing core, a prefetch instruction memory storing the prefetch program, a prediction logic module configured to generate prediction results based on the predictive computations, and a command generator configured to generate the read commands.
4. The system of claim 3, wherein the prefetch program includes logic for performing simple arithmetic and logical operations to predict the required data, the simple arithmetic and logical operations including linear transformations or similarity comparisons.
5. The system of claim 1, wherein the prefetch processor operates in parallel with the core computing units, such that the prefetch processor performs predictive computations and data prefetching while the core computing units execute the main workload.

6. The system of claim 1, wherein the prefetch processor is configurable with different prefetch programs to support different types of main workloads requiring data prefetching.
7. The system of claim 2, wherein the prefetch program is configured to predict activated experts based on input token features of the MoE model, including computing similarity between the input token features and expert-specific feature vectors.
8. A method of optimizing data prefetching for a parallel processing unit, comprising: executing a prefetch program on a dedicated prefetch processor integrated within the parallel processing unit to perform predictive computations; predicting data required for subsequent execution of core computing units in the parallel processing unit based on the predictive computations; issuing read commands from the prefetch processor to transfer the predicted data from host memory to high-bandwidth memory (HBM) of the parallel processing unit; and executing the main workload on the core computing units using the prefetched data stored in HBM.
9. The method of claim 8, wherein the main workload is inference of a Mixture of Experts (MoE) model, and the predicted data is parameters of experts predicted to be activated during inference.
10. The method of claim 8, wherein the predictive computations include analyzing input data related to the main workload to determine the data that will be required for subsequent execution.
11. The method of claim 8, further comprising operating the prefetch processor in parallel with the core computing units, such that predictive computations and data prefetching are performed concurrently with the execution of the main workload.
12. The method of claim 9, wherein predicting the activated experts includes computing similarity between input token features and expert-specific feature vectors, and selecting a subset of experts with the highest similarity scores.
13. A parallel processing unit comprising core computing units configured to execute main workloads, high-bandwidth memory (HBM) configured to store data for the core computing units, and a dedicated prefetch processor integrated within the parallel processing unit; wherein the prefetch processor is configured to execute a lightweight prefetch program to predict data required for the core computing units, issue read commands to prefetch the predicted data from host memory to the HBM, and operate independently of the core computing units and a host processor during prefetching.